

南开大学

汇编语言与逆向技术课程实验报告

Lab7: Reverse Engineering Challenge



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

目录

1 实验时间	2
2 实验目的	2
3 实验环境	2
4 实验流程	2
5 反汇编代码图形化视窗	2
6 逆向分析详细过程	3
6.1 计算过程与分支结构	3
6.2 challenge 条件判断	5
6.2.1 输入格式要求	5
6.3 solution 条件判断	5
6.3.1 输入格式要求	5
6.3.2 条件一验证	6
6.3.3 条件二验证	6
6.3.4 条件三验证	6
6.3.5 条件四验证	7
6.4 数据结构	8
6.5 代码求解	8
7 实验结果	9
8 附录：用于求解的完整 python 代码	9

1 实验时间

第 14 周星期一 9-10 节

2 实验目的

1. 熟悉反编译工具 Binary Ninja;
2. 熟悉反汇编代码的逆向分析过程;
3. 掌握反汇编语言中的数学计算、数据结构、条件判断、分支结构的识别和逆向分析

3 实验环境

Binary Ninja、PyCharm

4 实验流程

1. 利用 Binary Ninja 打开 challenge.exe 文件
2. 调出反汇编代码图形化视窗并分析整体分支架构和各块代码功能
3. 编写 python 程序得到 solution

5 反汇编代码图形化视窗

使用 Binary Ninja 打开待分析的 PE 文件 challenge.exe

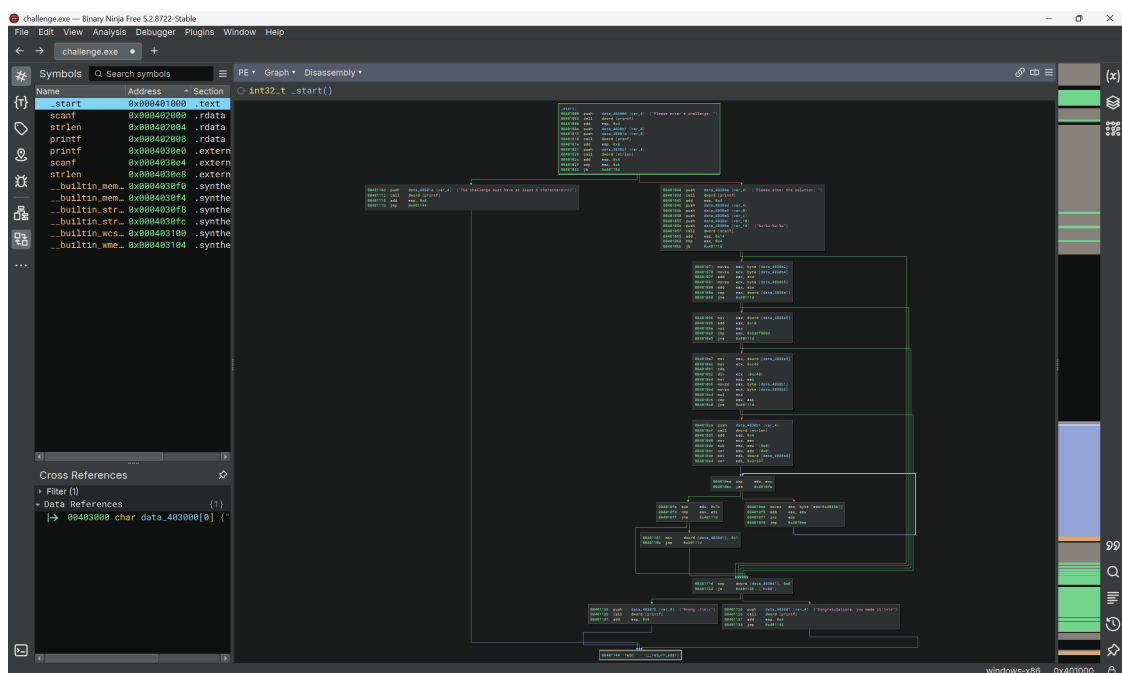


图 5.1: 反汇编代码图形化视窗

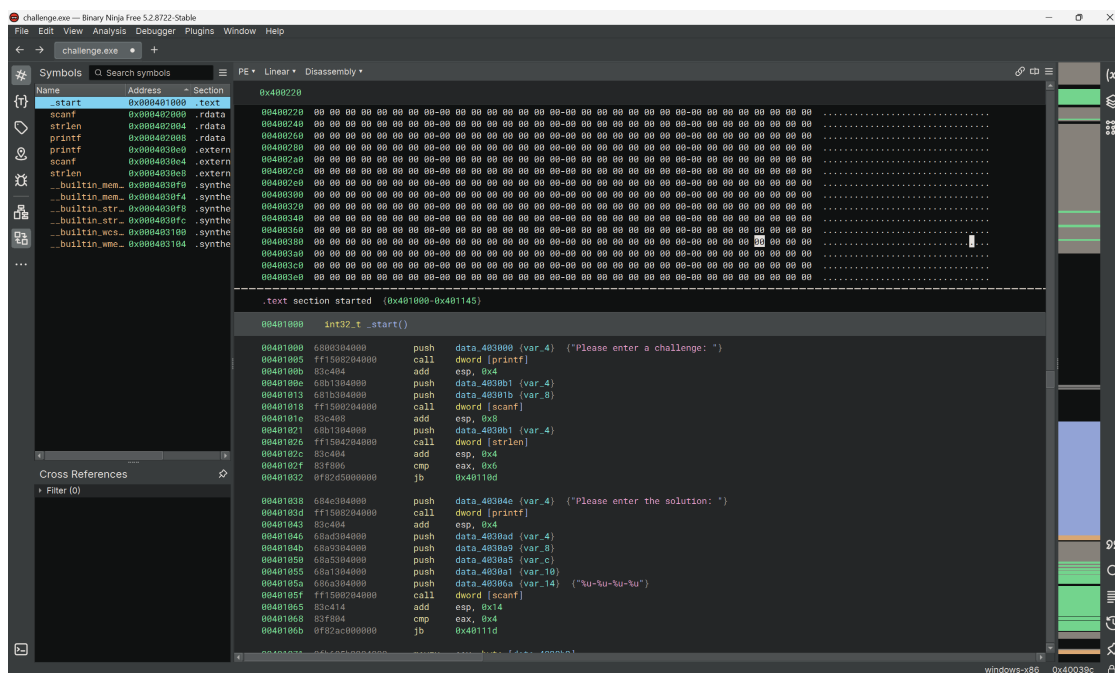


图 5.2: 反汇编代码

6 逆向分析详细过程

该题为逆向工程典型的 KeygenMe 题，要求用户输入一个 challenge，再输入一个符合特定格式的 solution（密钥）。程序内部有一套验证算法，只有满足所有条件的 solution 才能通过。

6.1 计算过程与分支结构

关于整体计算运行过程和分支结构，程序开始，要求输入 challenge 字符串和 solution。

首先程序会对 challenge 格式输入进行检查，通过继续程序，不通过直接输出错误提示，程序结束。

之后对 solution 格式判断和四个条件检查，有一个不通过则输出 Wrong，全部通过获得 “Congratulations, you made it!” 输出。

整体流程可用流程图表示：

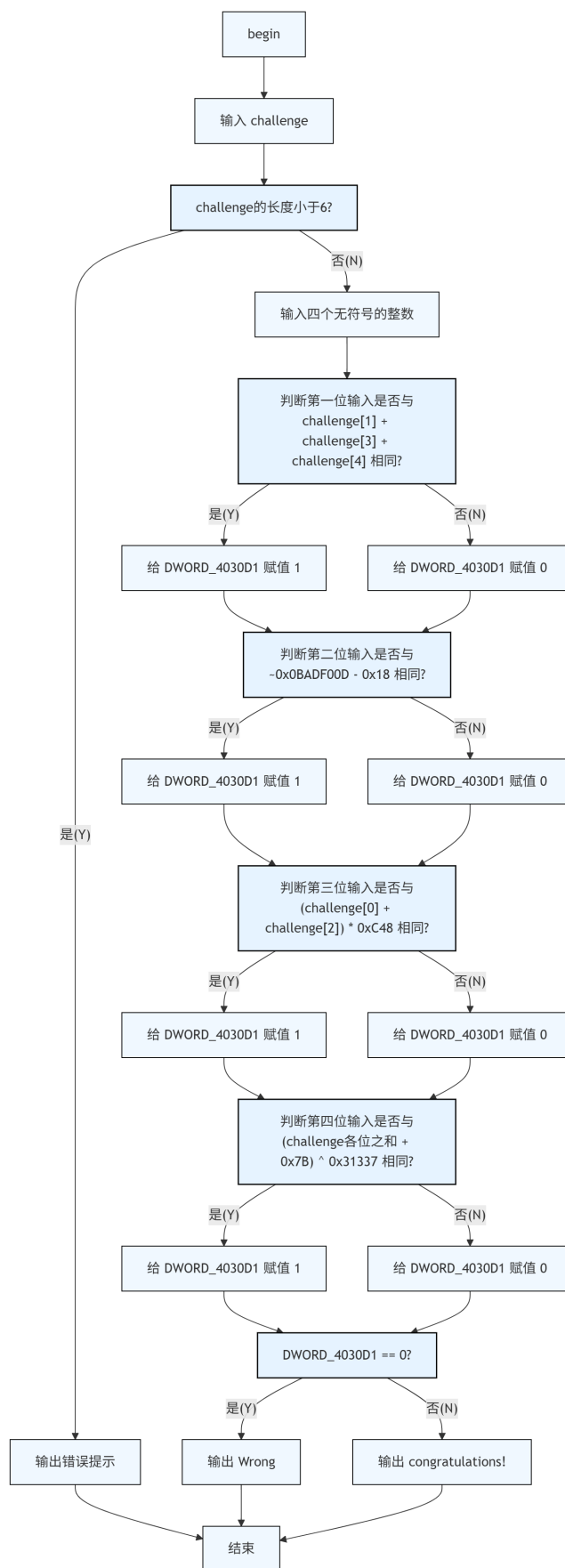


图 6.3: 程序逻辑示意图

6.2 challenge 条件判断

6.2.1 输入格式要求

```
1  _start:
2  00401000 push  data_403000 {var_4} {"Please enter a challenge: "}
3  00401005 call  dword [printf]      ; 显示提示信息
4  0040100b add   esp, 0x4             ; 清理栈空间
5  0040100e push  data_4030b1 {var_4} ; 推送参数2
6  00401013 push  data_40301b {var_8} ; 推送参数1
7  00401018 call  dword [scanf]      ; 调用scanf读取用户输入
8  0040101e add   esp, 0x8             ; 清理栈空间
9  00401021 push  data_4030b1 {var_4} ; 推送参数: 输入字符串地址
10 00401026 call  dword [strlen]     ; 调用strlen计算字符串长度
11 0040102c add   esp, 0x4             ; 清理栈空间
12 0040102f cmp   eax, 0x6             ; 比较长度值与6
13 00401032 jb    0x40110d            ; 如果长度<6则跳转到错误处理
```

检查 challenge 输入的长度是否小于 6。

- 是：直接输出错误提示，流程结束。
- 否：流程继续，要求用户输入四个无符号整数。

6.3 solution 条件判断

6.3.1 输入格式要求

solution 输入格式要求由 4 个无符号整数用“-”分隔。

```
1 00401046 push  data_4030ad {var_4} ; 推送第5个参数: 第4个无符号整数的存储地址
2 0040104b push  data_4030a9 {var_8} ; 推送第4个参数: 第3个无符号整数的存储地址
3 00401050 push  data_4030a5 {var_c} ; 推送第3个参数: 第2个无符号整数的存储地址
4 00401055 push  data_4030a1 {var_10} ; 推送第2个参数: 第1个无符号整数的存储地址
5 0040105a push  data_40306a {var_14} {"%u-%u-%u-%u"} ;
   格式字符串, 要求4个无符号整数用"-"分隔
6 0040105f call  dword [scanf]      ; 调用scanf读取格式化的输入
7 00401065 add   esp, 0x14           ; 清理栈空间 (5个参数 = 20字节)
```

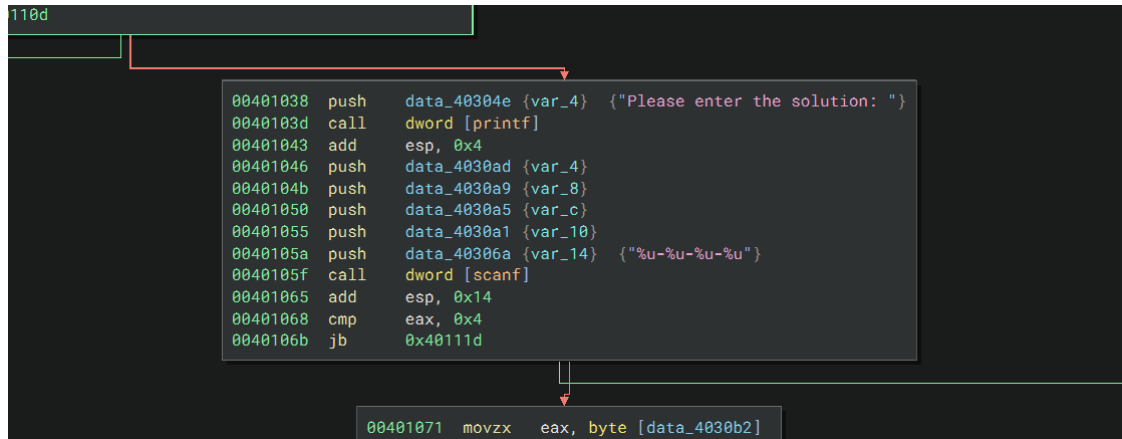


图 6.4: 代码在图形化视窗的部分

6.3.2 条件一验证

条件一要求用户输入的第一个无符号整数等于 challenge 字符串的第 2, 4, 5 个字符之和。

```

1 00401088 add    eax, ecx    ; eax = challenge[1] + challenge[3] + chal
2 0040108a cmp    eax, dword [data_4030a1] ; 比较上面的和与solution的第一个无符号整数
3 00401090 jne    0x40111d ; 如果不相等, 跳转到错误处理

```

需满足:

$$challenge[1] + challenge[3] + challenge[4] == solution[0]$$

6.3.3 条件二验证

条件二要求第二个无符号整数必须等于 1159790554

```

1 00401096 mov    eax, dword [data_4030a5] ; 将solution的第二个整数加载到eax
2 0040109b add    eax, 0x18                ; eax = solution[1] + 0x18 (24)
3 0040109e not    eax                      ; eax = ~(solution[1] + 0x18)
4                                ; NOT操作: 按位取反
5 004010a0 cmp    eax, 0xbadf000d          ; 比较取反后的值是否等于0xbadf000d
6 004010a5 jne    0x40111d                ; 如果不相等, 跳转到错误处理

```

将第二位数字加上 18h 然后按位取反, 和 BADF000Dh 进行比较, 如果不相等, 跳转到错误处理。

6.3.4 条件三验证

条件三要求将 solution 的第一位无符号整数乘以一个计算出的值, 其结果必须等于 (solution 的第三位无符号整数 / 0xC48) 的商。

具体过程如下:

```

1 004010a7 mov    eax, dword [data_4030a9] ; 加载 solution 的第三个整数到 eax
2 004010ac mov    ecx, 0xc48                ; ecx = 0xC48 (十进制 3144)
3 004010b1 cdq                                ; 将 eax 符号扩展为 edx:eax, 为除法做准备

```

```

4 004010b2 div     ecx                ; edx:eax / ecx, 商保存在 eax, 余数在 edx
5 004010b4 mov     esi, eax           ; 将商保存到 esi
6 004010b6 movzx   eax, byte [data_4030b1] ; 零扩展加载 data_4030b1 处的字节到 eax
7 004010bd movzx   ecx, byte [data_4030b3] ; 零扩展加载 data_4030b3 处的字节到 ecx
8 004010c4 mul     ecx                ; eax = eax * ecx (无符号乘法)
9 004010c6 cmp     eax, esi           ; 比较: 因子 是否等于商 (solution[2] / 0xC48)
10 004010c8 jne    0x40111d           ; 如果不相等, 跳转到错误处理

```

设:

- $S_1 = \text{data_4030a9}$ (输入的第三个整数)
- $F_1 = \text{data_4030b1}$ (一个字节, 来自固定数据区)
- $F_2 = \text{data_4030b3}$ (一个字节, 来自固定数据区)

则条件三要求:

$$(F_1 \times F_2) = \left\lfloor \frac{S_1}{3144} \right\rfloor$$

其中 $\lfloor \cdot \rfloor$ 表示整数除法 (商)。必须精确相等。

6.3.5 条件四验证

将 solution 的第四位无符号整数与 0x31337 进行异或, 然后减去 0x7B, 其结果必须等于 data_4030b1 字符串中所有字符的 ASCII 值之和。

```

1 004010de mov     edi, dword [data_4030ad] ; 加载 solution 的第四位整数到 edi
2 004010e4 xor     edi, 0x31337             ; edi = solution[3] XOR 0x31337

```

```

1 ; 循环开始: 计算字符串所有字符的 ASCII 值之和
2 004010ea cmp     edx, ecx                ; 比较 i (edx) 与字符串长度 (ecx)
3 004010ec jae     0x4010fa                ; 如果 i 长度, 跳出循环
4 004010ee movzx   ebx, byte [edx+0x4030b1] ; 取字符串第 i 个字符
5 004010f5 add     eax, ebx                ; 累加字符 ASCII 值到 eax
6 004010f7 inc     edx                    ; i++
7 004010f8 jmp     0x4010ea                ; 跳回循环开头

```

```

1 004010fa sub     edi, 0x7b                ; edi = (solution[3] XOR 0x31337) - 0x7B
2 004010fd cmp     eax, edi                ;
   比较: 字符串字符和 (eax) 是否等于计算值 (edi)
3 004010ff jne     0x40111d                ; 如果不相等, 跳转到错误处理

```

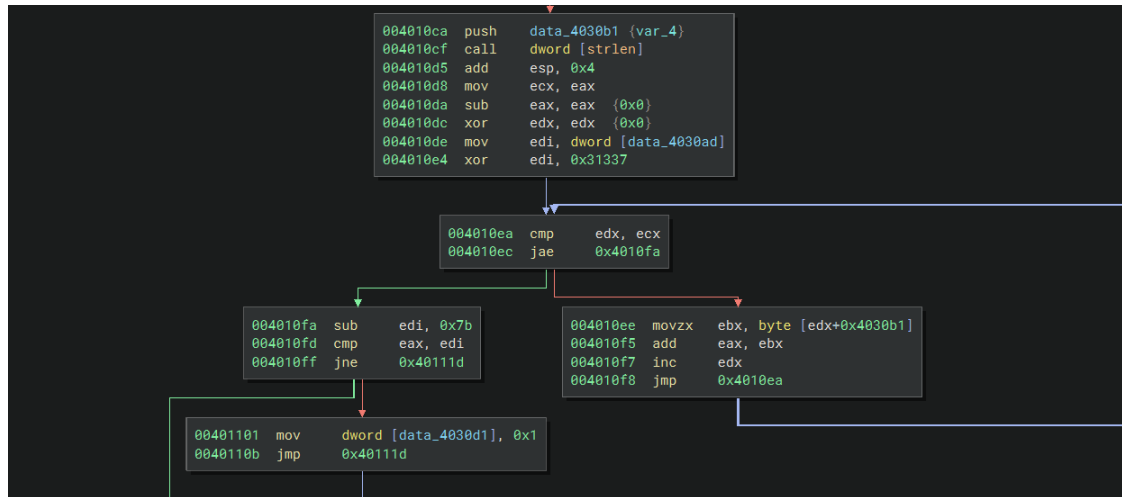



图 6.5: 代码在图形化视窗的部分

6.4 数据结构

该汇编程序涉及到字符串常量、无符号整数变量、整数数组..... 等多种汇编元素。

6.5 代码求解

利用 python 语言编写程序解决。下图是我利用 python 代码得到字符串”abcdef” 的密钥的过程

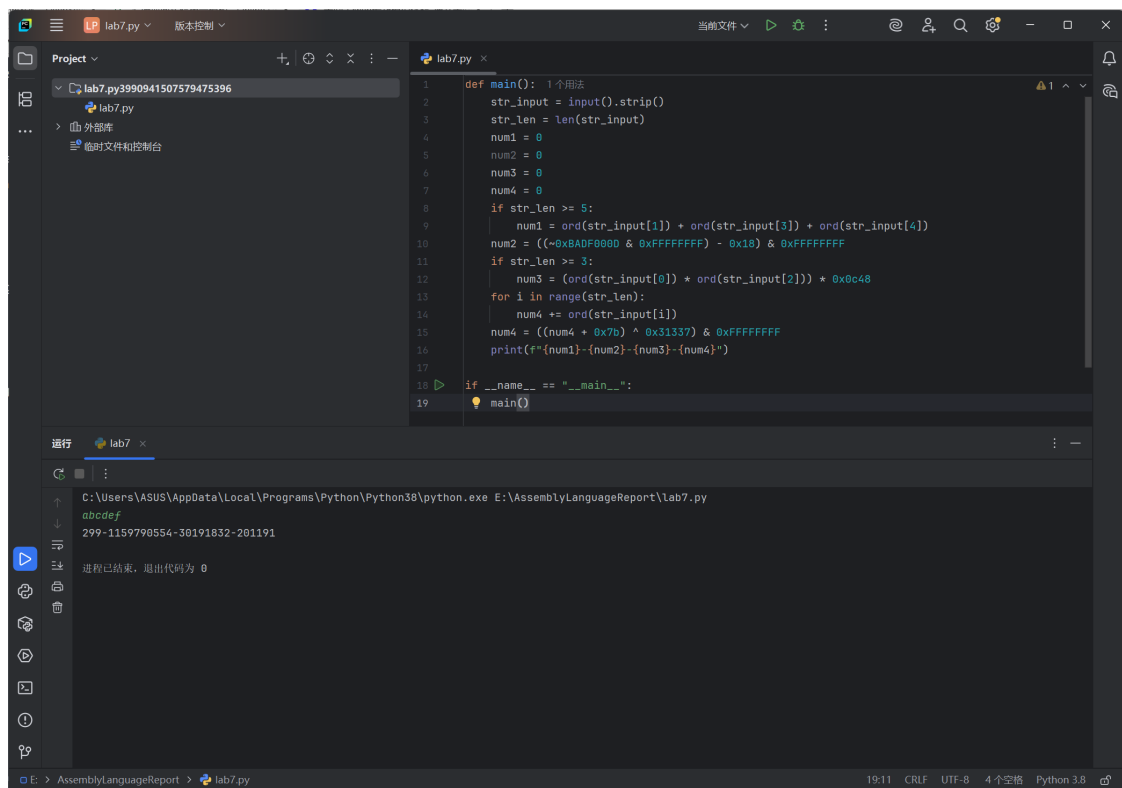
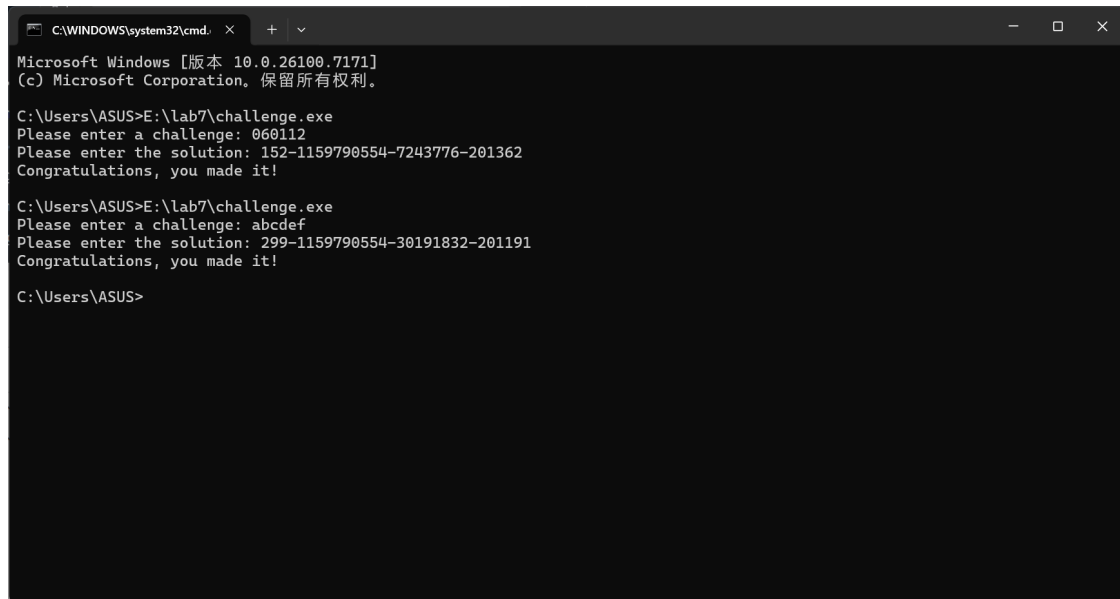


图 6.6: 程序运行截图

7 实验结果

这里输入了”060112” 和”abcdef” 两组。全部通过获得 “Congratulations, you made it!” 输出。



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [版本 10.0.26100.7171]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\ASUS>E:\lab7\challenge.exe
Please enter a challenge: 060112
Please enter the solution: 299-1159790554-7243776-201362
Congratulations, you made it!

C:\Users\ASUS>E:\lab7\challenge.exe
Please enter a challenge: abcdef
Please enter the solution: 299-1159790554-30191832-201191
Congratulations, you made it!

C:\Users\ASUS>
```

图 7.7: 实验结果截图

8 附录：用于求解的完整 python 代码

```
1 def main():
2     str_input = input().strip()
3     str_len = len(str_input)
4     num1 = 0
5     num2 = 0
6     num3 = 0
7     num4 = 0
8     if str_len >= 5:
9         num1 = ord(str_input[1]) + ord(str_input[3]) + ord(str_input[4])
10        num2 = ((~0xBADF000D & 0xFFFFFFFF) - 0x18) & 0xFFFFFFFF
11        if str_len >= 3:
12            num3 = (ord(str_input[0]) * ord(str_input[2])) * 0x0c48
13        for i in range(str_len):
14            num4 += ord(str_input[i])
15        num4 = ((num4 + 0x7b) ^ 0x31337) & 0xFFFFFFFF
16        print(f"{num1}-{num2}-{num3}-{num4}")
17
18 if __name__ == "__main__":
19     main()
```